

NOTES ON MATHEMATICAL AESTHETICS

# A Cronenberg Creature

Geometric Deformation, Scalar Fields, and Generative Body-Horror



Cesar Galindo

Universidad Panamericana, Ciudad UP Bosque Real

**Abstract.** This note gives a polished version of the full construction: a philosophical reading of identity, replacement, and nihilism; a geometric grammar for organic deformation; scalar-field and level-set models; a wavelet anatomy of the rendered body; ASCII and color renderers in C; pseudocode for a generative pipeline; stochastic and optimization interpretations; and a final original Cronenberg-inspired monster. The central thesis is that a creature can be produced from a controlled sequence of operations: perturb a recognizable body, break its symmetry, fuse local centers, distribute identity across scales, contaminate the surface with texture, and render the result as an image.

April 26, 2026

## Contents

|    |                                                     |    |
|----|-----------------------------------------------------|----|
| 1  | Prelude: identity, replacement, and deformation     | 1  |
| 2  | Cronenberg grammar                                  | 2  |
| 3  | The base geometry                                   | 2  |
| 4  | Symmetry breaking                                   | 4  |
| 5  | Fusion through scalar fields                        | 4  |
| 6  | Layer anatomy                                       | 5  |
| 7  | Wavelet anatomy: the monster as a multiscale object | 7  |
| 8  | Parameters as artistic controls                     | 10 |
| 9  | Excursion sets and threshold bodies                 | 10 |
| 10 | Temporal deformation, colonies, and composition     | 10 |
| 11 | Rendering pseudocode                                | 12 |
| 12 | ASCII rendering in C                                | 13 |
| 13 | Color rendering in C                                | 14 |
| 14 | Design coordinates                                  | 15 |
| 15 | Final Cronenberg creature                           | 15 |
| 16 | Design brief for the final image                    | 16 |
| 17 | Stochastic interpretation                           | 17 |
| 18 | Genetic algorithms and learning dynamics            | 18 |
| 19 | Pedagogical uses                                    | 19 |
| 20 | Reproducibility                                     | 19 |

# 1 Prelude: identity, replacement, and deformation

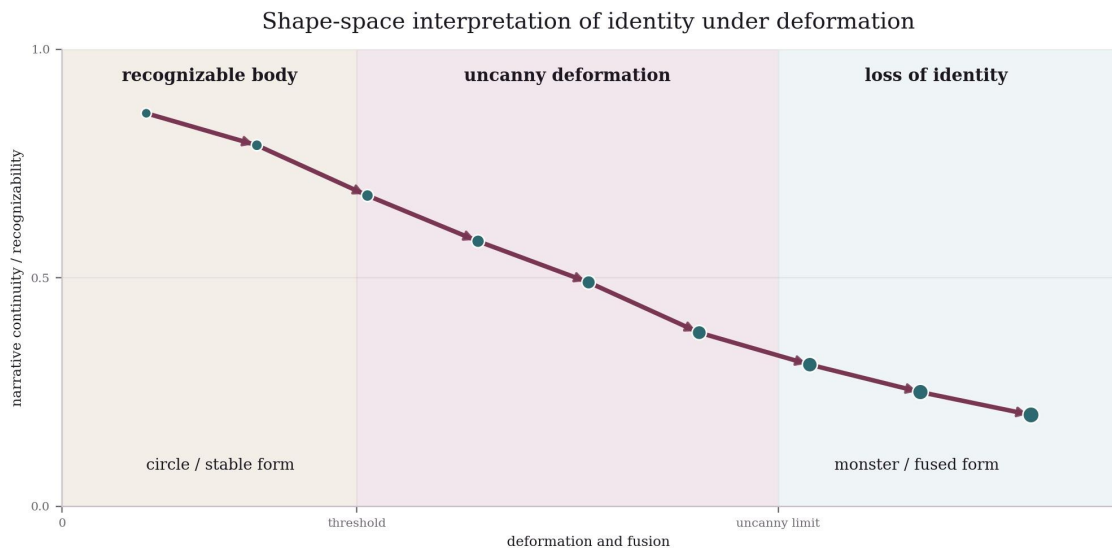
A Cronenberg-style mutation is philosophically effective because it does not merely add monstrosity to the world. It makes the body unreliable. The face that should identify a person becomes a surface of variation; the boundary that should separate one organism from another becomes negotiable; the social role that should anchor an identity can suddenly be occupied by a replacement. The horror is therefore ontological before it is visual: the question is not only “what happened to this body?” but also “what remains the same after the body changes?”

The mathematical language of this note treats identity as persistence under deformation. A shape is not preserved by staying rigid; it is preserved by moving through a space of possible forms while keeping enough structure to remain recognizable. This is why a creature must not be pure noise. If all invariants disappear, there is no uncanny effect: there is only visual entropy. The disturbing regime lies between recognition and collapse. A silhouette remains body-like, but no longer trustworthy. A face is almost visible, but it is not where it should be. A limb is almost functional, but its orientation is wrong. In that intermediate region, deformation becomes a philosophical event.

There are three levels of identity in the construction. The first is *geometric*: a curve can be continuously deformed without tearing. The second is *phenotypic*: an observer still reads the object as a body, a face, a mouth, or a limb. The third is *narrative*: a form continues to count as the same creature because its trajectory of changes can be followed. These levels need not agree. A replacement may preserve the role while breaking the lived continuity; a mutation may preserve the topology while destroying the familiar appearance; a narrative may insist on sameness after the body has already become another thing.

This is also where nihilism enters. At the scale of a large state space, a failed configuration can be replaced by a nearby configuration with similar macroscopic observables. At the human scale, however, the discontinuity is not erased by similarity. The subject who buries a possible self, or sees a body become an object, learns that replaceability is not the same as insignificance. The global structure may say that many worlds, bodies, or images are available; the local experience still says that this one mattered.

The artistic problem can now be stated mathematically. We want an object that travels from stable recognition to unstable fusion without becoming arbitrary. Equations supply the deformation; scalar fields supply the body; thresholds supply membranes and cavities; texture supplies organic contingency. The output is a creature because it is not only drawn. It has a grammar.



**Figure 1:** Shape-space interpretation of the philosophical problem. The useful aesthetic interval is not the stable body and not total loss, but the middle region where recognizability survives deformation.

**Remark 1.1 (The grotesque as failed recognition)** *The grotesque is structured failure. It asks the viewer to recognize something and then denies the comfort of that recognition. In the present model, this failure is produced by a controlled mismatch between boundary, center, symmetry, and texture.*

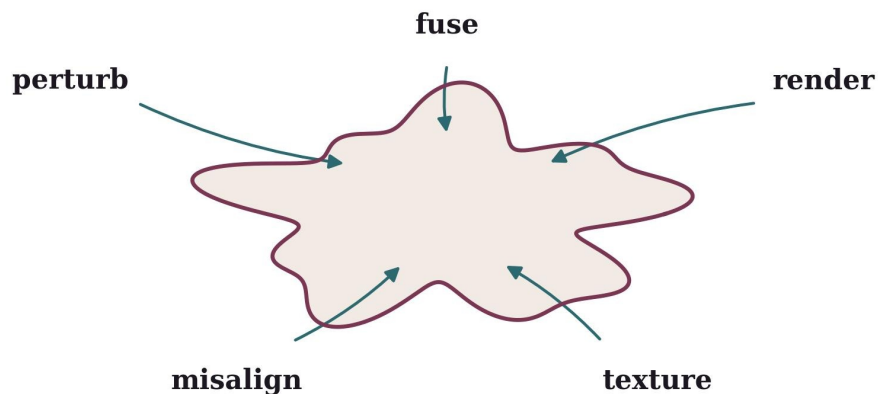
## 2 Cronenberg grammar

The word “Cronenberg” is used here as shorthand for a body-horror language in which biology is unstable, surfaces are ambiguous, and identity migrates through flesh. The point is not to imitate a specific film, monster, or frame. The point is to formalize a visual syntax: an original creature is built by operations that have both mathematical and aesthetic content.

The grammar has five verbs. First, one *perturbs* a stable form so that the circle becomes a body-like outline. Second, one *misaligns* the form by breaking a symmetry that the viewer expected. Third, one *fuses* several centers so that organs, faces, mouths, or masses appear to share the same torso. Fourth, one *textures* the surface so that the image stops looking mechanical and begins to look biological. Finally, one *renders* the scalar field through color, contrast, and threshold choices. The creature is not any single layer; it is the accumulated effect of these verbs.

This grammar is useful because it avoids two weak extremes. On one side, a purely regular image is too legible and therefore not monstrous. On the other side, an uncontrolled image is too chaotic and therefore not memorable. A Cronenberg-inspired design needs legibility under stress. Its silhouette should remain iconic, but the interior should betray the silhouette. It should be readable at a distance and disturbing at close range.

### Cronenberg grammar as a generative sequence



The grammar is a sequence of verbs, not a checklist: every operation changes how the next one is perceived.

**Figure 2:** The grammar as a sequence of verbs. The figure deliberately avoids a boxed checklist: each operation modifies how the next operation is perceived.

The rest of the note develops this grammar from equations to code. We begin with a boundary, pass to scalar fields, read level sets as bodies, and end with a final creature image whose anatomical features can be traced back to the same operations.

## 3 The base geometry

Let  $\mathbf{T} = \mathbf{R}/2\pi\mathbf{Z}$ . The elementary organism is a closed polar curve

$$r(\theta) = 1 + a \sin(k\theta) + b \sin(m\theta + \phi), \quad \theta \in \mathbf{T}. \quad (1)$$

The parameters have direct visual meaning. The amplitudes  $a, b$  control the size of the lobes; the frequencies  $k, m$  control how many protrusions appear; the phase  $\phi$  prevents the curve from being too

symmetrical. The unit circle is not discarded. It remains the hidden reference body from which mutation is measured.

**Definition 3.1 (Harmonic creature boundary)** *A harmonic creature boundary is the image of*

$$\gamma(\theta) = r(\theta)(\cos \theta, \sin \theta), \quad (2)$$

where  $r$  is given by (1) and remains strictly positive.

**Proposition 3.1 (Regularity)** *Assume  $r(\theta) > 0$  for all  $\theta$ , and  $r^2(\theta) + r'(\theta)^2 > 0$  for all  $\theta$ . Then  $\gamma$  is a smooth immersed closed curve.*

*Proof.* Since  $r$  is trigonometric,  $\gamma$  is smooth. Differentiating,

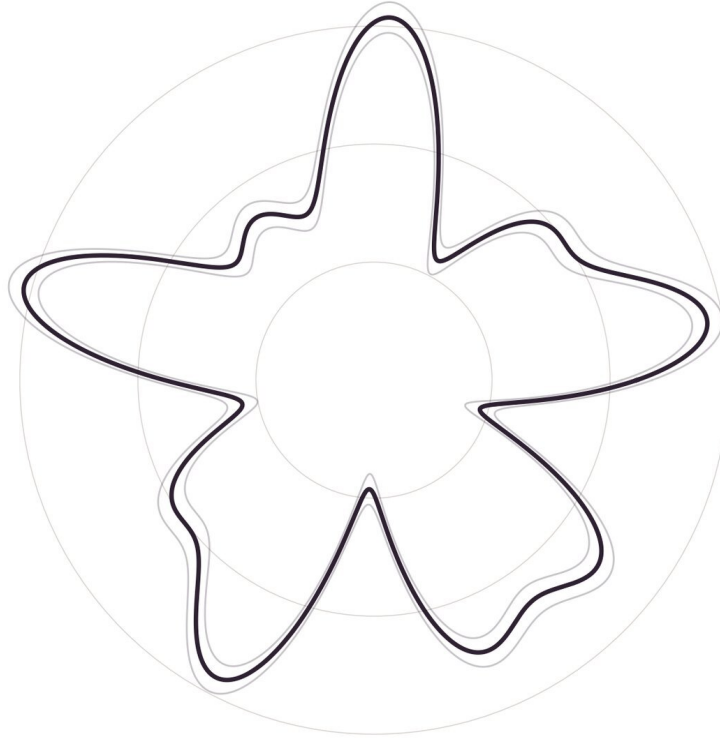
$$\gamma'(\theta) = r'(\theta)(\cos \theta, \sin \theta) + r(\theta)(-\sin \theta, \cos \theta).$$

The two vectors on the right are orthonormal, hence

$$\|\gamma'(\theta)\|^2 = r'(\theta)^2 + r(\theta)^2.$$

The stated hypothesis makes this non-zero everywhere. Thus  $\gamma$  is an immersion. Periodicity gives a closed curve. ■

Polar boundary: harmonic deformation



**Figure 3:** The base curve is a circle made unstable by two harmonics. Neighboring curves indicate how a small normal displacement already suggests organic thickness.

The first figure is intentionally modest: it shows that the monster begins as a deformation of something very simple. This is important pedagogically. The creature is not born from visual complexity; it is born from a small number of controlled parameters.

## 4 Symmetry breaking

A nearly bilateral object is legible. A perfectly chaotic object is often merely noisy. The uncanny object lies between the two: it almost has a symmetry, but the symmetry fails at the moment when the viewer begins to trust it. In the formula, the phase  $\phi$  is the small dial that moves the object into this regime.

**Lemma 4.1 (A Fourier symmetry test)** *For  $r(\theta) = 1 + a \sin(k\theta) + b \sin(m\theta + \phi)$ , the equality*

$$r(\pi - \theta) = r(\theta)$$

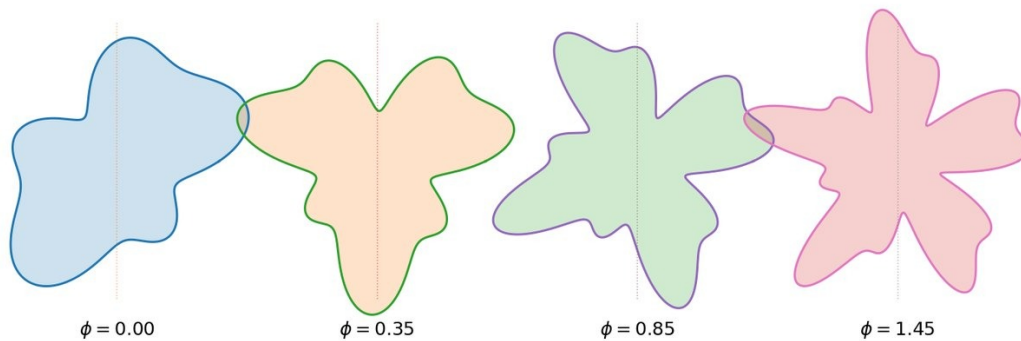
*for all  $\theta$  imposes arithmetic constraints on  $k, m$  and phase constraints on  $\phi$ . Generic values of  $\phi$  break the symmetry.*

*Proof.* The identity

$$\sin(k(\pi - \theta)) = \sin(k\pi - k\theta)$$

expands into coefficients of  $\sin(k\theta)$  and  $\cos(k\theta)$ . Equality for all  $\theta$  forces the coefficients of each independent Fourier mode to match. The same argument applies to the  $m$ -mode. Since  $\phi$  changes the coefficients of  $\sin(m\theta)$  and  $\cos(m\theta)$ , a generic phase violates the required identities. ■

Controlled symmetry breaking: from near-bilateral to uncanny



**Figure 4:** Controlled symmetry breaking. The phase parameter  $\phi$  moves the boundary from near-bilateral recognition to uncanny asymmetry.

This is one of the central aesthetic principles of the document. Symmetry is not simply removed; it is wounded. The viewer still detects the ghost of order, and that is precisely why the distortion feels bodily rather than ornamental.

## 5 Fusion through scalar fields

Boundaries produce silhouettes; scalar fields produce bodies. Let  $c_i = (x_i, y_i) \in \mathbf{R}^2$ , weights  $\alpha_i > 0$ , and  $\varepsilon > 0$ . Define the metaball field

$$M(x, y) = \sum_{i=1}^N \frac{\alpha_i}{(x - x_i)^2 + (y - y_i)^2 + \varepsilon}. \quad (3)$$

Nearby centers merge smoothly in the level sets of  $M$ . This is the mathematical model of fusion: several local centers begin to share one body.



**Definition 5.1 (Generative creature field)** *Let*

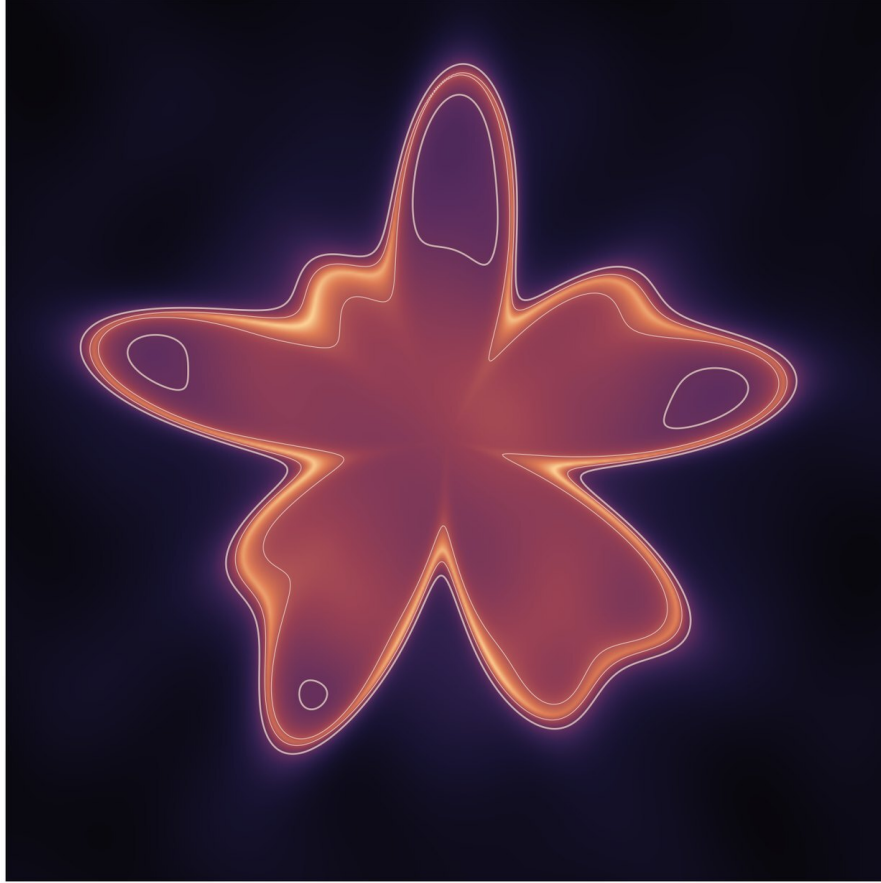
$$\begin{aligned} B(x, y) &= \exp\{-\lambda|\rho - r(\theta)|\}, \\ I(x, y) &= \frac{1}{1 + \exp\{\beta(\rho - r(\theta))\}}, \\ M(x, y) &= \sum_{i=1}^N \frac{\alpha_i}{(x - x_i)^2 + (y - y_i)^2 + \varepsilon}, \\ T(x, y) &= \text{a deterministic or random texture field.} \end{aligned}$$

where  $\rho = \sqrt{x^2 + y^2}$  and  $\theta = \text{atan2}(y, x)$ . A generative creature field is

$$\Phi(x, y) = c_B B(x, y) + c_I I(x, y) + c_M \tanh M(x, y) + \eta T(x, y). \quad (4)$$

The rendered creature is obtained by applying a color map to  $\Phi$  and drawing selected level sets.

Organic field: body, fusion, and texture



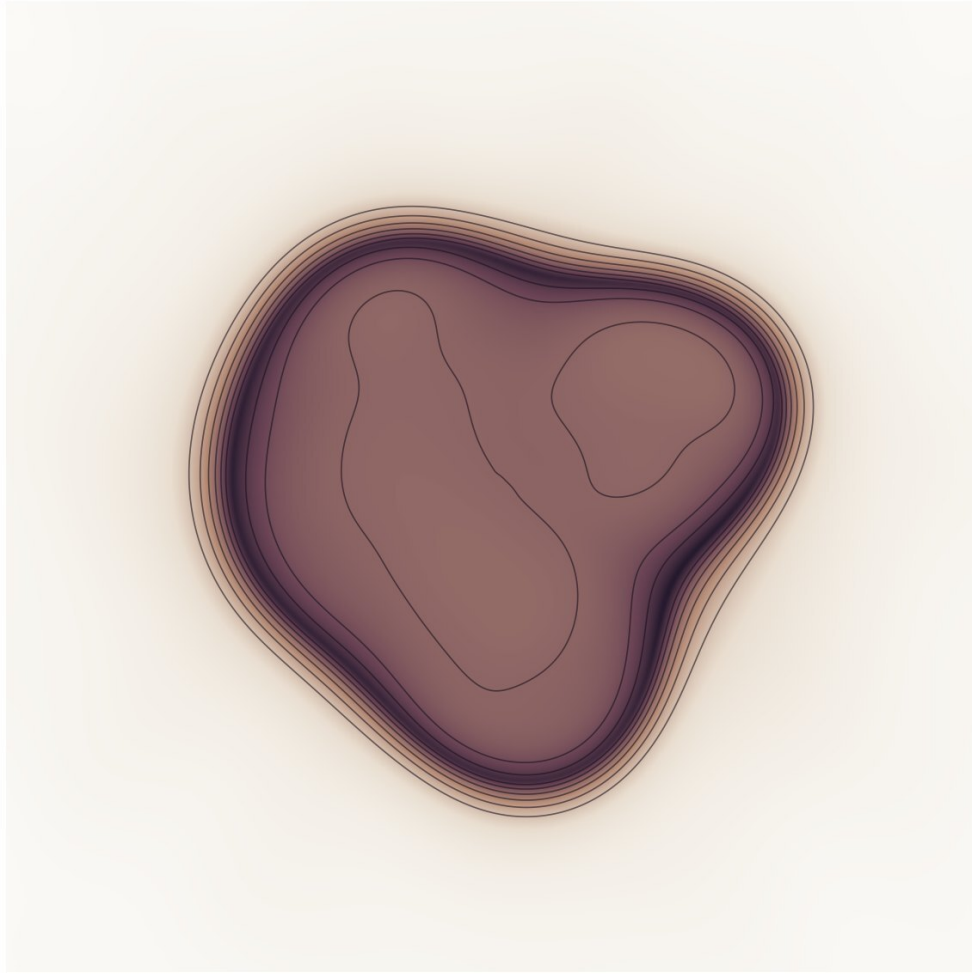
**Figure 5:** A full creature field. Boundary, interior, fusion, and texture combine into a single scalar surface.

The shift from curve to field is the decisive mathematical move. Once the creature is a field, cavities, mouths, membranes, eyes, and organs can be read as level-set phenomena. They are not pasted onto the body; they emerge from the same continuous surface.

## 6 Layer anatomy

Equation (4) is additive, but its visual meaning is sequential. The boundary term gives the viewer an edge to follow; the interior term gives mass; the metaball term creates centers of attraction; the texture

## Metaball fusion: level sets of a scalar field

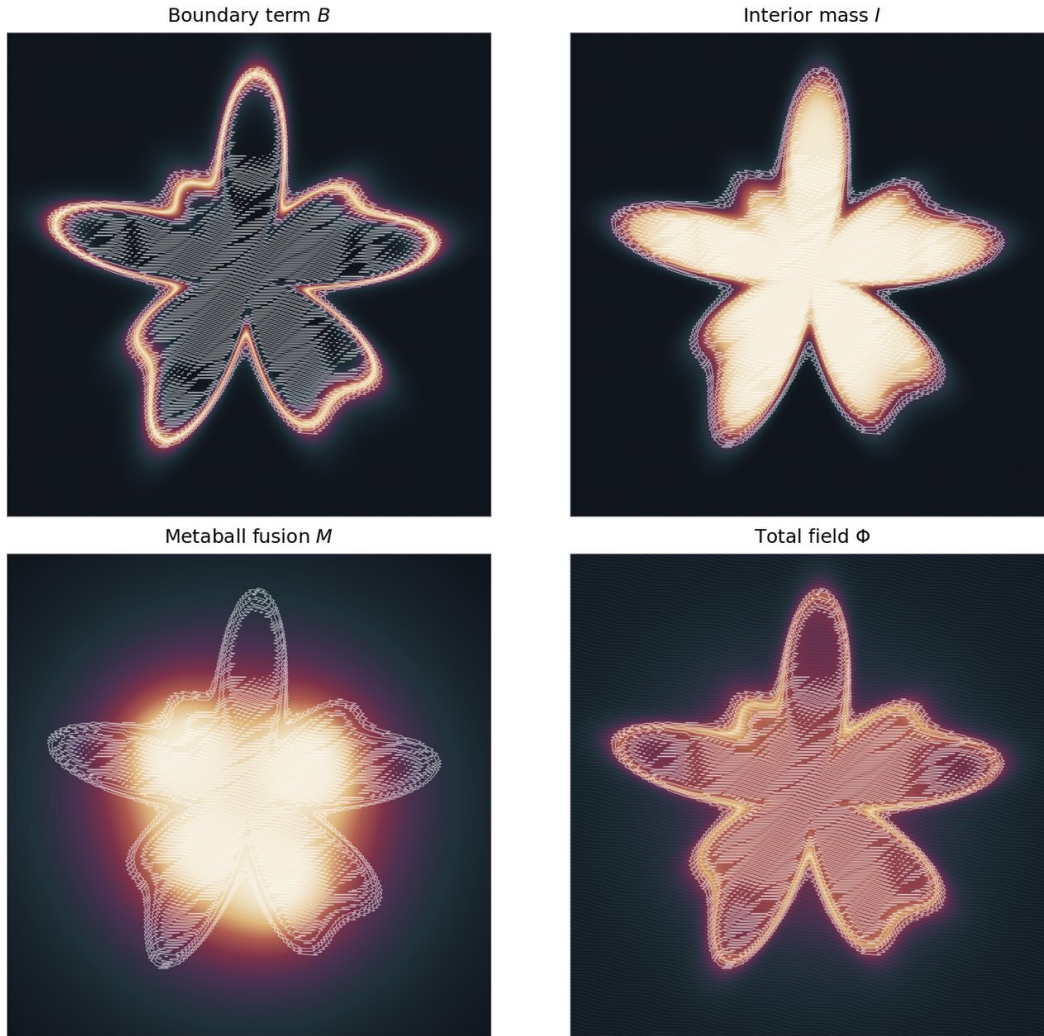


**Figure 6:** Metaball fusion as level-set geometry. Several local centers are visible, but outer level curves merge into a common organism.

term contaminates the surface with irregularity. The final image is a negotiated compromise among these layers.

This decomposition also explains why the construction is teachable. Each layer can be switched off, studied, and reintroduced. The creature therefore behaves like an experiment rather than a single opaque image.





**Figure 7:** Layer decomposition of  $\Phi$ . The same body is built from a boundary term, an interior term, a fusion term, and the final scalar field.

## 7 Wavelet anatomy: the monster as a multiscale object

The scalar field  $\Phi$  should not be read as a flat image. It is a stratified object. Its identity is distributed across scales: the largest scales carry posture and silhouette; intermediate scales carry organs, cavities, embedded mouths, and local fusions; the finest scales carry veins, scratches, wet folds, teeth, pores, and surface instability. This is precisely the regime in which wavelet analysis becomes natural.

Fourier modes are global: changing one coefficient changes the whole body. Wavelets, by contrast, are local and multiscale. They allow us to ask where a deformation occurs and at which scale it occurs. This is a better language for body horror, because the grotesque is rarely a global oscillation. It is a local failure of form: a mouth opens in the wrong region, an eye appears inside a shoulder mass, a membrane tears only near a fold, or a tendon repeats at smaller and smaller scales.

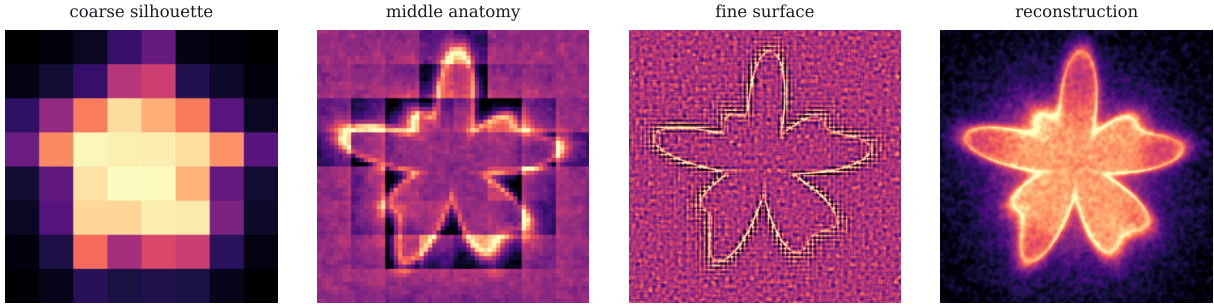
Let  $D \subset \mathbf{R}^2$  be the image domain and regard  $\Phi$  as an element of  $L^2(D)$ . For an orthonormal two-dimensional wavelet system, one may write formally

$$\Phi(x, y) = \sum_k a_{J,k} \varphi_{J,k}(x, y) + \sum_{j \geq J} \sum_k \sum_{\nu \in \{h,v,d\}} c_{j,k}^\nu \psi_{j,k}^\nu(x, y),$$

where

$$a_{J,k} = \langle \Phi, \varphi_{J,k} \rangle, \quad c_{j,k}^\nu = \langle \Phi, \psi_{j,k}^\nu \rangle.$$

The scaling coefficients  $a_{J,k}$  record the low-frequency organism. The wavelet coefficients  $c_{j,k}^\nu$  record horizontal, vertical, and diagonal detail at location  $k$ , scale  $j$ , and orientation  $\nu$ .



**Figure 8:** Wavelet anatomy of the creature field. The coarse component carries the residual body; the middle bands carry anatomical deformation; the fine bands carry surface instability; their sum reconstructs the rendered scalar field.

This gives a sharper philosophical statement. At low frequency the monster remains a body. At intermediate frequency it becomes mutation: organs, cavities, and fused centers begin to compete with the original silhouette. At high frequency it becomes event, wound, noise, and surface. Wavelets therefore formalize the passage from identity to mutation. The creature is not identical to any one scale; it is the negotiation among them.

**Definition 7.1 (Wavelet energy and multiscale fingerprint)** For a rendered creature field  $\Phi$ , define the detail energy at scale  $j$  by

$$E_j = \sum_{\nu \in \{h,v,d\}} \sum_k |c_{j,k}^\nu|^2.$$

Together with the coarse energy

$$E_{\text{coarse}} = \sum_k |a_{J,k}|^2,$$

the vector

$$\mathcal{E}(\Phi) = (E_{\text{coarse}}, E_J, E_{J+1}, E_{J+2}, \dots)$$

is the multiscale fingerprint of the creature. After normalization,

$$p_j = \frac{E_j}{\sum_\ell E_\ell}, \quad H(\Phi) = - \sum_j p_j \log p_j$$

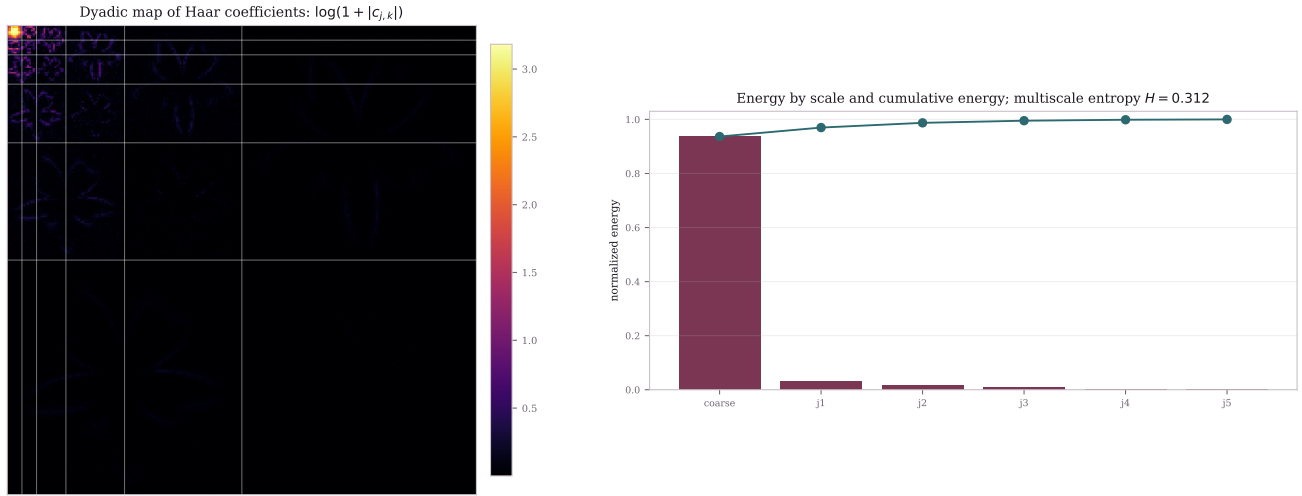
measures how broadly the image distributes energy across scales.

**Proposition 7.1 (Energy bookkeeping)** For an orthonormal wavelet basis and  $\Phi \in L^2(D)$ , the total  $L^2$ -energy decomposes as

$$\|\Phi\|_2^2 = \sum_k |a_{J,k}|^2 + \sum_{j \geq J} \sum_k \sum_{\nu \in \{h,v,d\}} |c_{j,k}^\nu|^2.$$

*Proof.* This is Parseval's identity applied to the orthonormal wavelet expansion of  $\Phi$ . The scaling functions account for the coarse approximation space, while the wavelets account for the orthogonal detail spaces. Orthogonality eliminates cross terms, so the squared norm is the sum of squared coefficients. ■

A smooth embryonic monster concentrates most of its energy in the coarse coefficients. A violent Cronenbergian monster transfers energy into middle and high scales. The middle bands give the body its distorted anatomy; the finest bands give the skin its threatening granularity. A useful design target is therefore not merely “high complexity.” It is a controlled spectrum: enough low-frequency energy to preserve a body, enough medium-frequency energy to deform it, and enough high-frequency energy to make the surface feel alive.

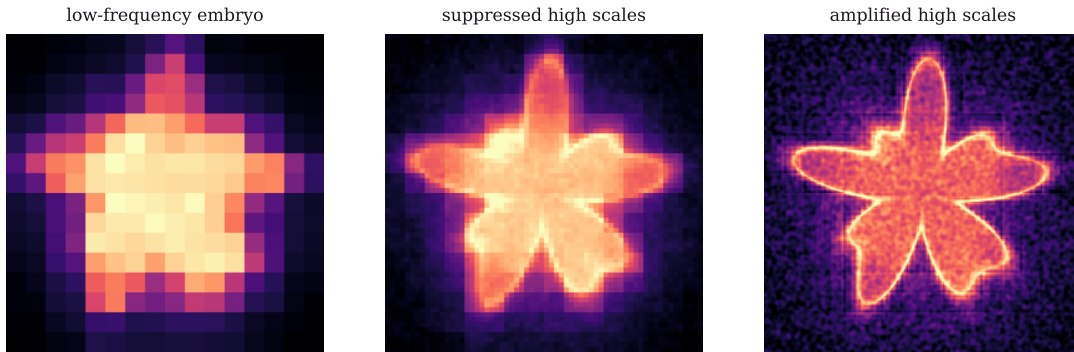


**Figure 9:** Left: dyadic map of Haar coefficients, showing where scale and location concentrate the field variation. Right: normalized energy by scale and cumulative energy. The large low-frequency mass explains recognizability; the non-zero detail bands explain deformation and surface disturbance.

The same decomposition is also a renderer. By multiplying coefficient bands by scale weights  $\alpha_j$ , one can suppress or amplify entire aesthetic regimes:

$$c_{j,k}^\nu \mapsto \alpha_j c_{j,k}^\nu, \quad \tilde{\Phi} = \text{IDWT}(a_{J,k}, \alpha_j c_{j,k}^\nu).$$

Small  $\alpha_j$  at high frequency makes the creature soft and embryonic. Large  $\alpha_j$  at high frequency produces torn surface, jagged membranes, and a more aggressive visual temperature. Thus wavelets do not merely analyze the monster after the fact; they provide knobs for generating it.



**Figure 10:** Coefficient editing as aesthetic control. Removing high scales produces a soft low-frequency organism; restoring them gives a readable body with texture; amplifying them pushes the field toward sharper wounds, folds, and surface violence.

**Listing 1:** Wavelet pipeline for analyzing and editing a creature field.

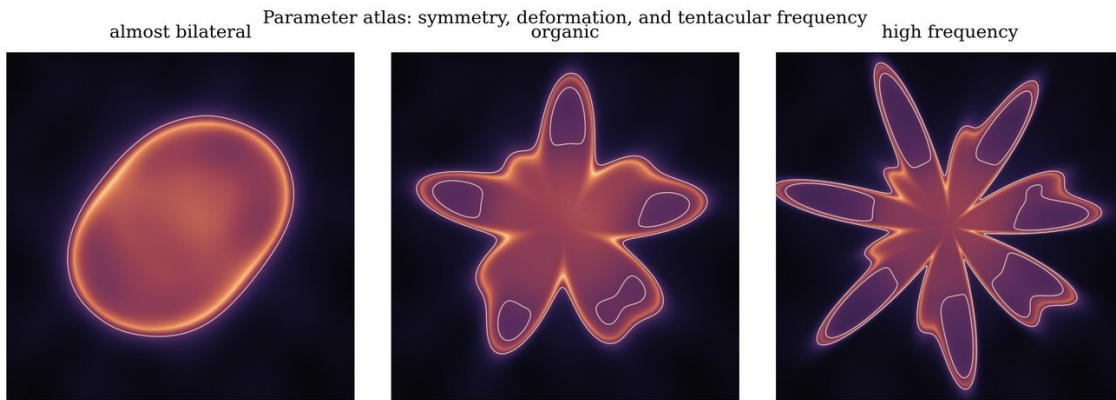
```
Phi <- generate_creature_field()
{a_J,k, c_j,k^nu} <- DWT(Phi)
E_j <- sum_{k,nu} |c_j,k^nu|^2,   p_j <- E_j/sum_l E_l
H <- -sum_j p_j log(p_j)
alpha_low, alpha_middle, alpha_high <- choose_aesthetic_weights()
c_j,k^nu <- alpha_j c_j,k^nu
Phi_tilde <- IDWT({a_J,k, alpha_j c_j,k^nu})
render Phi_tilde as color field, level set, or texture layer
```

This is why wavelets belong in the main body of the article. They do not interrupt the story; they complete it. The earlier sections explain how to make a creature. The wavelet section explains how the creature is internally organized: not as one object at one resolution, but as a hierarchy of recognitions and failures of recognition.

## 8 Parameters as artistic controls

The model is useful because its parameters are both mathematical and visual. A change in  $k$  changes the Fourier spectrum, but it also changes the number of teeth, lobes, or tendrils directions. A change in  $\eta$  changes the amplitude of a texture field, but it also changes whether the surface reads as plastic, skin, or tissue.

| Parameter  | Mathematical role       | Visual effect                                    |
|------------|-------------------------|--------------------------------------------------|
| $a, b$     | Fourier amplitudes      | Size of lobes and organic protrusions            |
| $k, m$     | Fourier frequencies     | Number of lobes, teeth, or tentacular directions |
| $\phi$     | Phase shift             | Controlled loss of symmetry                      |
| $\lambda$  | Boundary sharpness      | Thickness of the visible contour                 |
| $\alpha_i$ | Metaball strengths      | Local centers of mass and fusion                 |
| $\eta$     | Texture amplitude       | Skin irregularity and surface noise              |
| $\alpha_j$ | Wavelet-band multiplier | Scale control: silhouette, anatomy, or skin      |
| Color map  | Scalar-to-RGB map       | Emotional temperature of the organism            |



**Figure 11:** A parameter atlas. The first panel stays close to bilateral recognition, the second moves to organic instability, and the third increases high-frequency deformation.

The atlas is the mathematical equivalent of a sketchbook. It makes visible that the creature is not a fixed object, but a family of possible objects indexed by a genome of parameters.

## 9 Excursion sets and threshold bodies

For a threshold  $u$ , define the visible body

$$K_u = \{(x, y) \in D : \Phi(x, y) \geq u\}, \quad (5)$$

where  $D \subset \mathbf{R}^2$  is the rendering window. Varying  $u$  changes what counts as body, membrane, tendril, or void. This makes the image a level-set object rather than a mere bitmap.

The threshold is a philosophical parameter as much as a numerical one. It decides what is allowed to count as the organism and what becomes background, wound, or emptiness.

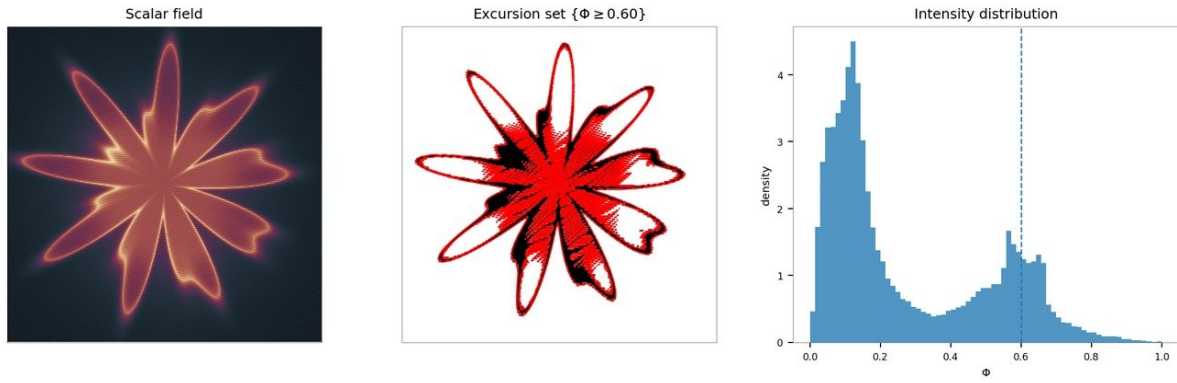
## 10 Temporal deformation, colonies, and composition

Animation appears by allowing the phase to depend on time:

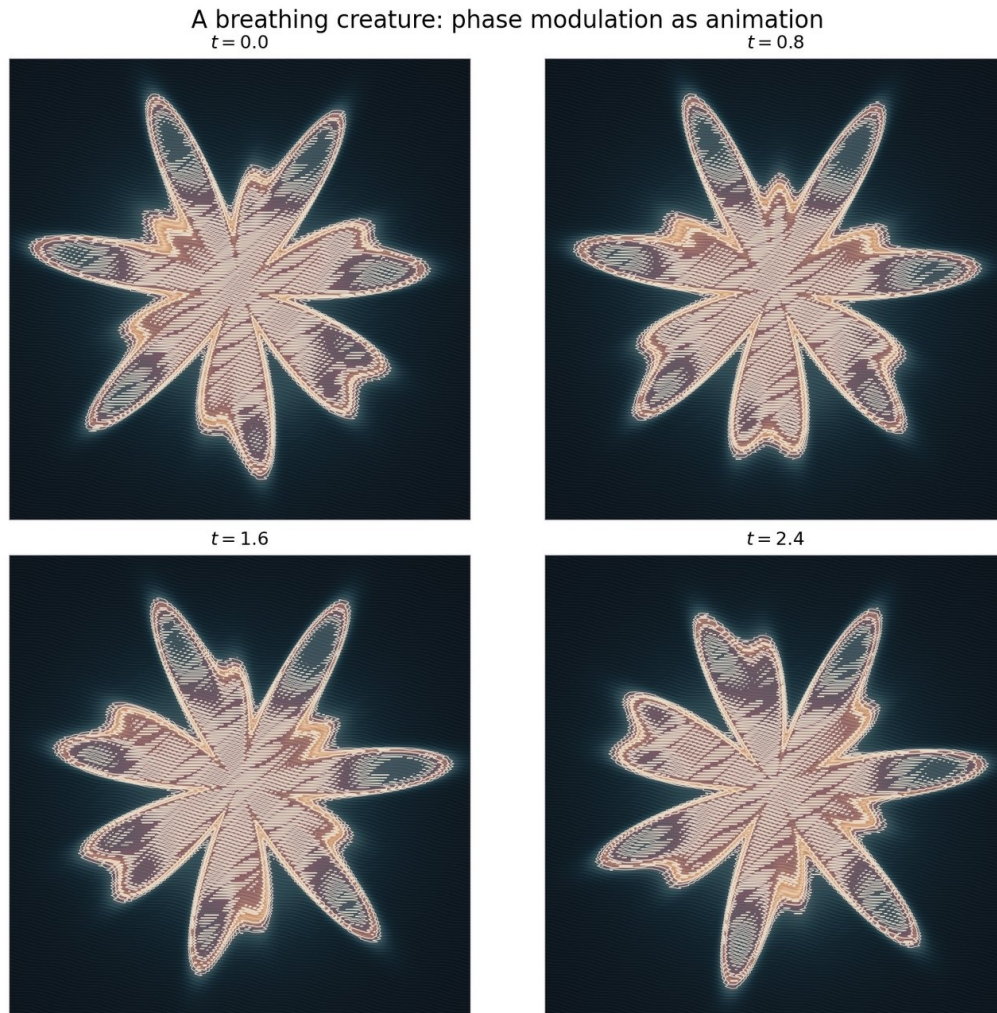
$$r_t(\theta) = 1 + a \sin(k\theta + \omega_1 t) + b \sin(m\theta + \phi - \omega_2 t).$$

This gives a breathing organism without changing the mathematical grammar. The identity of the creature is the trajectory of the parameters, not a single still frame.





**Figure 12:** From scalar field to body. A threshold selects an excursion set, and the histogram records how much of the image lives at each intensity scale.



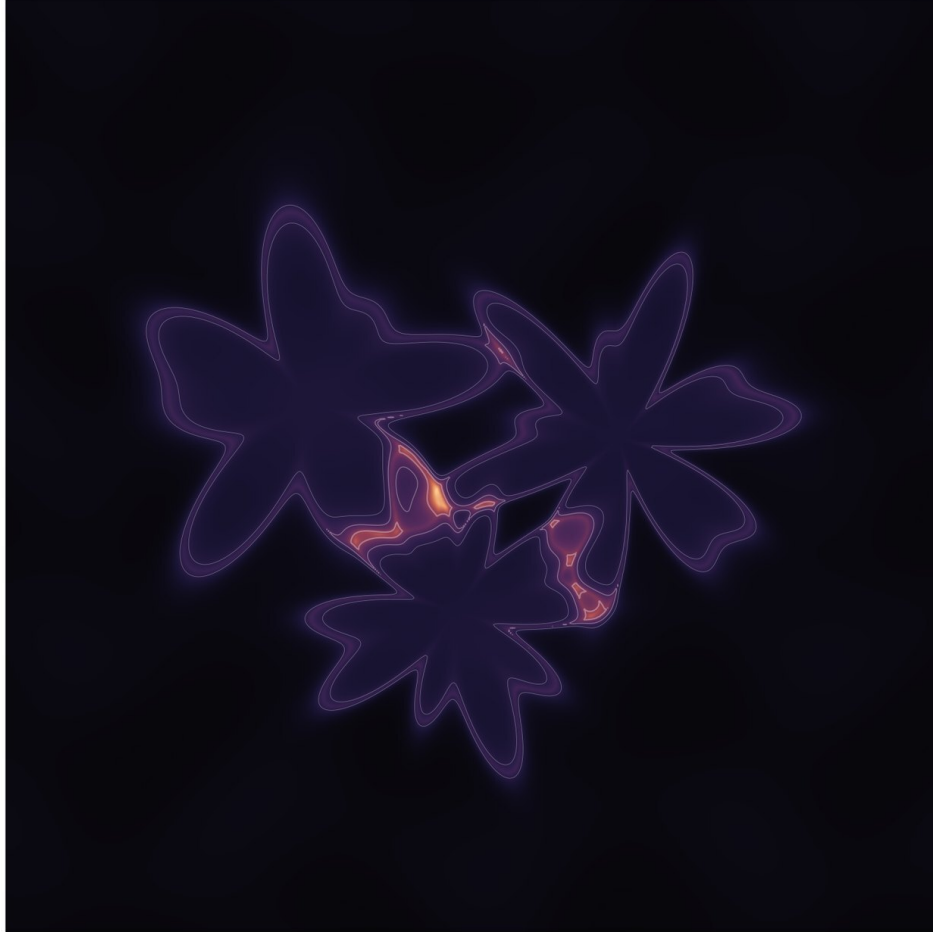
**Figure 13:** A breathing creature. Time is introduced by phase modulation; the organism moves without losing its generative identity.

A single organism is one scalar field. A colony is a superposition:

$$\Phi_{\text{colony}}(x, y) = \sum_{j=1}^J w_j \Phi_j(A_j(x, y) - b_j),$$

where  $A_j$  is a rotation-scaling matrix and  $b_j$  is a translation. The same local grammar now produces ecological composition.

Colony composition: superposition of deformed fields



**Figure 14:** A colony obtained by superposing shifted and rescaled creature fields.

Time and colony formation are two ways of escaping the single-object view. One makes the creature into a process; the other makes it into a population.

## 11 Rendering pseudocode

The algorithm is a pixel discretization of (4). Each pixel is mapped into the plane, the field is evaluated, and the scalar value is converted into either a character or a color.

**Listing 2:** Pixel renderer for a generative creature field.

```
for each pixel (i,j):
    x,y  = map_to_plane(i,j)
    theta = atan2(y,x)
    rho  = sqrt(x*x + y*y)

    rd    = 1 + a*sin(k*theta) + b*sin(m*theta + phi)
    boundary = exp(-lambda*abs(rho - rd))
```



```

interior = 1/(1 + exp(beta*(rho - rd)))

fusion = 0
for each center c_l=(x_l,y_l):
    fusion += alpha_l / ((x-x_l)^2 + (y-y_l)^2 + epsilon)

texture = noise(x,y)
value    = cB*boundary + cI*interior + cM*tanh(fusion) + eta*texture

output(i,j) = color_map(value)

```

## 12 ASCII rendering in C

The first program renders the boundary field into ASCII. It is pedagogically useful because it exposes rasterization without graphics libraries.

**Listing 3:** ASCII renderer. Compile with `gcc creature_ascii.c -o creature_ascii -lm`.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  #define WIDTH 120
6  #define HEIGHT 60
7
8  static double a = 0.35, b = 0.20;
9  static int k = 5, m = 9;
10 static double phi = 0.50;
11 static double scale = 1.50;
12
13 static double r_deform(double theta) {
14     return 1.0 + a * sin(k * theta) + b * sin(m * theta + phi);
15 }
16
17 static double field(double x, double y) {
18     double r = sqrt(x * x + y * y);
19     double theta = atan2(y, x);
20     double rd = r_deform(theta);
21     double d = fabs(r - rd);
22     return exp(-10.0 * d);
23 }
24
25 static void render_ascii(void) {
26     const char *gradient = " .:-+*#%@";
27     const int levels = 10;
28
29     for (int j = 0; j < HEIGHT; ++j) {
30         for (int i = 0; i < WIDTH; ++i) {
31             double x = (2.0 * i / (double) WIDTH - 1.0) * scale;
32             double y = (2.0 * j / (double) HEIGHT - 1.0) * scale;
33             double val = field(x, y);
34             int idx = (int) floor(val * (levels - 1));
35             if (idx < 0) idx = 0;
36             if (idx >= levels) idx = levels - 1;
37             putchar(gradient[idx]);
38         }
39         putchar('\n');
40     }
41 }
42
43 int main(void) {
44     render_ascii();
45     return EXIT_SUCCESS;
46 }

```

## 13 Color rendering in C

The second program writes a plain PPM image. The format is deliberately simple: a header followed by RGB triples. This makes the code portable and easy to inspect in a first algorithms course.

**Listing 4:** Color PPM renderer. Compile with `gcc creature_color_ppm.c -o creature_color_ppm -lm`.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  #ifndef M_PI
6  #define M_PI 3.14159265358979323846
7  #endif
8
9  #define W 700
10 #define H 700
11 #define NUM_BLOBS 4
12
13 static double scale = 1.85;
14
15 static double r_deform(double theta) {
16     return 1.0 + 0.35 * sin(5.0 * theta) + 0.20 * sin(9.0 * theta + 0.60);
17 }
18
19 static double hash2(int x, int y) {
20     unsigned int n = (unsigned int)(x * 374761393u + y * 668265263u);
21     n = (n ^ (n >> 13)) * 1274126177u;
22     n = n ^ (n >> 16);
23     return (double)(n & 0xffu) / 255.0;
24 }
25
26 static double smooth_noise(double x, double y) {
27     int xi = (int) floor(x);
28     int yi = (int) floor(y);
29     double xf = x - xi;
30     double yf = y - yi;
31
32     double v00 = hash2(xi, yi);
33     double v10 = hash2(xi + 1, yi);
34     double v01 = hash2(xi, yi + 1);
35     double v11 = hash2(xi + 1, yi + 1);
36
37     double sx = xf * xf * (3.0 - 2.0 * xf);
38     double sy = yf * yf * (3.0 - 2.0 * yf);
39     double a = v00 * (1.0 - sx) + v10 * sx;
40     double b = v01 * (1.0 - sx) + v11 * sx;
41     return a * (1.0 - sy) + b * sy;
42 }
43
44 static double metaballs(double x, double y) {
45     const double centers[NUM_BLOBS][3] = {
46         { 0.40, 0.20, 0.17},
47         {-0.50, 0.30, 0.16},
48         { 0.20, -0.60, 0.15},
49         {-0.30, -0.40, 0.13}
50     };
51     double sum = 0.0;
52     for (int i = 0; i < NUM_BLOBS; ++i) {
53         double dx = x - centers[i][0];
54         double dy = y - centers[i][1];
55         double d2 = dx * dx + dy * dy + 0.025;
56         sum += centers[i][2] / d2;
57     }
58     return sum;
59 }
60
61 static double field(double x, double y) {
62     double r = sqrt(x * x + y * y);
63     double theta = atan2(y, x);
64     double rd = r_deform(theta);

```

```

65     double body = 0.65 * exp(-9.0 * fabs(r - rd));
66     double interior = 0.40 / (1.0 + exp(12.0 * (r - rd)));
67     double fusion = 0.38 * tanh(metaballs(x, y));
68     double skin = 0.14 * smooth_noise(3.0 * x, 3.0 * y);
69     return body + interior + fusion + skin;
70 }
71
72 static void color_map(double v, int *R, int *G, int *B) {
73     if (v < 0.0) v = 0.0;
74     if (v > 1.0) v = 1.0;
75     *R = (int)(255.0 * pow(v, 1.20));
76     *G = (int)(180.0 * sin(M_PI * v));
77     *B = (int)(178.0 * (1.0 - v) + 80.0 * v);
78 }
79
80 static void render_ppm(const char *filename) {
81     FILE *fp = fopen(filename, "w");
82     if (!fp) {
83         perror("fopen");
84         exit(EXIT_FAILURE);
85     }
86     fprintf(fp, "P3\n%d %d\n255\n", W, H);
87
88     /* First pass: estimate maximum for a stable normalization. */
89     double maxv = 0.0;
90     for (int j = 0; j < H; ++j) {
91         for (int i = 0; i < W; ++i) {
92             double x = (2.0 * i / (double) W - 1.0) * scale;
93             double y = (2.0 * j / (double) H - 1.0) * scale;
94             double v = field(x, y);
95             if (v > maxv) maxv = v;
96         }
97     }
98
99     for (int j = 0; j < H; ++j) {
100         for (int i = 0; i < W; ++i) {
101             double x = (2.0 * i / (double) W - 1.0) * scale;
102             double y = (2.0 * j / (double) H - 1.0) * scale;
103             double v = field(x, y) / maxv;
104             int R, G, B;
105             color_map(v, &R, &G, &B);
106             fprintf(fp, "%d %d %d ", R, G, B);
107         }
108         fputc('\n', fp);
109     }
110     fclose(fp);
111 }
112
113 int main(void) {
114     render_ppm("creature_color.ppm");
115     return EXIT_SUCCESS;
116 }

```

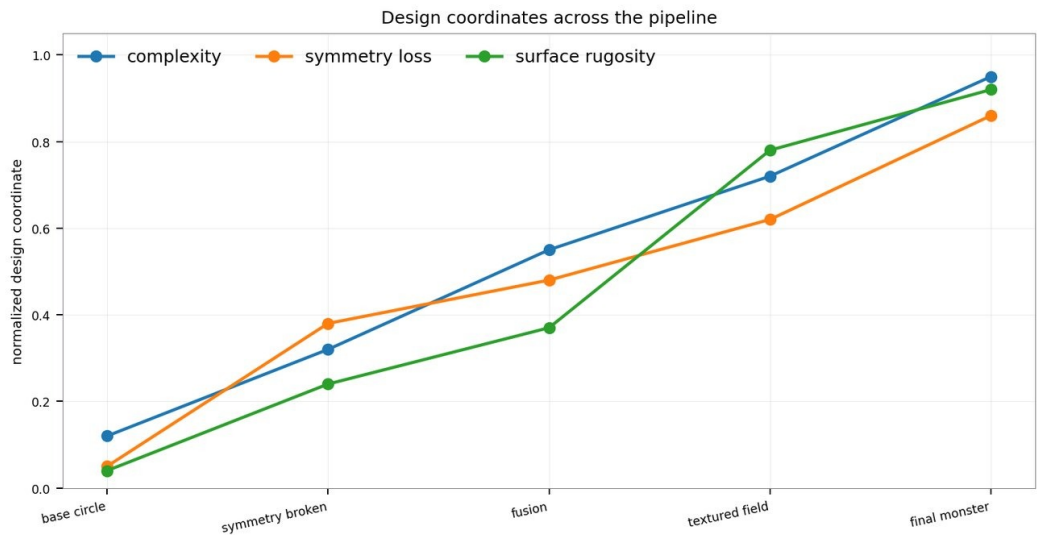
## 14 Design coordinates

The whole construction can be summarized by three visible design coordinates—complexity, symmetry loss, and surface rugosity—complemented by the wavelet energy diagnostic. The mathematical model starts as a curve and ends as a textured organism whose identity is distributed across wavelet bands.

These coordinates are not meant to replace the equations. They are a compact way to talk about the image after it has been rendered: how much structure remains, how much symmetry has failed, and how rough the surface has become.

## 15 Final Cronenberg creature

The final concept image translates the mathematical grammar into a single readable monster. Its composition is not random. The silhouette comes from boundary deformation; the uneven mass distribution



**Figure 15:** Design coordinates across the pipeline. Complexity, symmetry loss, and rugosity rise as the model moves from circle to monster.

comes from symmetry breaking; the multiple facial and oral centers come from metaball fusion; the wet folds and mottled color come from texture; the cavities and embedded features come from threshold geometry.

| Mathematical operation | Visual translation in the final monster                           |
|------------------------|-------------------------------------------------------------------|
| Boundary deformation   | Irregular silhouette, hooked limbs, protrusions                   |
| Symmetry breaking      | One side carries more mass and more tendrils                      |
| Metaball fusion        | Multiple facial and oral centers share a torso                    |
| Texture field          | Veins, folds, wet surface variation, mottled color                |
| Threshold geometry     | Cavities, mouths, eyes, and membranes emerge as level-set regions |
| Temporal deformation   | The pose suggests an organism caught mid-mutation                 |

The anatomy overlay is included to make the artistic image accountable to the mathematical construction. It says, in effect: here is where the formula became silhouette, here is where centers fused, here is where texture overran the boundary.

16 Design brief for the final image

The final image was generated from a design brief rather than copied from an existing creature. The brief can be summarized as follows.

Create one original Cronenberg-inspired monster as the central subject. It should have asymmetrical anatomy, fused organic masses, multiple embedded faces or mouths, extra eyes, tendon-like tendrils, bone-like protrusions, stretched skin, wet texture, strong silhouette, dramatic rim lighting, and a dark laboratory or industrial atmosphere. The result should remain readable and iconic, not visually muddy.

The point of including this brief is methodological. It shows how mathematical descriptors can be translated into image-generation language. The formal terms “asymmetry,” “fusion,” “texture,” “surface variation,” and “silhouette” survive the passage from equations to visual design.



**Figure 16:** Final original Cronenberg-inspired monster. The image is the visual endpoint of the generative grammar: an asymmetrical, fused, organic body with embedded centers and high surface rugosity.

## 17 Stochastic interpretation

If  $T$  is replaced by a random field  $T_\omega$ , then  $\Phi_\omega$  becomes a random field on  $\mathbf{R}^2$ . The visible creature at threshold  $u$  is the random compact set

$$K_u(\omega) = \{(x, y) \in D : \Phi_\omega(x, y) \geq u\}. \quad (6)$$

This connects the construction to random level sets, excursion sets, and random media.

One may also introduce an energy functional on curves,

$$\mathcal{E}(\gamma) = \int_0^{2\pi} \left[ \frac{\kappa}{2} |\gamma''(\theta)|^2 + V(\gamma(\theta)) - \sigma \Phi(\gamma(\theta)) \right] d\theta. \quad (7)$$

A low-energy boundary prefers to travel along high-intensity regions of the creature field, while curvature





**Figure 17:** Annotated anatomy of the final monster. Labels identify the residual mask, shoulder mass, embedded eyes, secondary jaw-field, abdominal mouths, tendrils, and hooked silhouette.

penalizes excessive oscillation. This is a clean bridge from generative art to stochastic analysis: the surface becomes a medium, the body becomes an excursion set, and the outline becomes an object selected by energy.

## 18 Genetic algorithms and learning dynamics

The parameter vector

$$\mathbf{g} = (a, b, k, m, \phi, \lambda, \beta, \alpha_1, \dots, \alpha_N, \eta)$$

is a genome. The image is the phenotype. Mutation changes  $\mathbf{g}$ , crossover mixes parameter families, and a fitness function scores the resulting creature.

**Listing 5:** Genetic search over creature parameters.

```
initialize population of genomes g_1, ..., g_P
```

```

for generation = 1,...,G:
  for each genome g_i:
    render creature image I_i
    compute features:
      symmetry_loss(I_i)
      contour_complexity(I_i)
      texture_rugosity(I_i)
      aesthetic_score(I_i)
    fitness_i = weighted combination of these features

  select parents with probability proportional to fitness
  apply crossover to selected parent genomes
  apply mutation to amplitudes, phases, centers, and texture strength
  keep the best elite genomes
return best creature genome and its rendered image

```

This is the bridge to mathematical foundations of learning dynamics. The optimization does not learn from data in the usual supervised sense; it explores a parameterized function space under a selection rule. Nevertheless, it already contains the basic ideas of representation, loss, mutation, search, and convergence diagnostics.

## 19 Pedagogical uses

The same construction can be taught at several levels. In calculus, students vary  $r(\theta)$ , compute  $r'$ , and connect critical points of the radius with protrusions of the image. In Fourier analysis, the creature is a finite harmonic expansion. In linear algebra, one can vectorize rendered images and build a PCA basis of creatures. In probability, the texture term turns the picture into a random field and the visible body into an excursion set. In algorithms, the parameter vector is a genome and the rendered image is a phenotype. In numerical methods, PPM rendering gives a minimal example of image generation without external libraries.

## 20 Reproducibility

The source folder contains the full LaTeX document, all generated figures, the wavelet figures, the C code for ASCII and PPM rendering, the final concept image, and the auxiliary figure-generation scripts. The C programs can be compiled with the following commands.

```

gcc creature_ascii.c -o creature_ascii -lm
gcc creature_color_ppm.c -o creature_color_ppm -lm

```

The PPM output can be converted to PNG with common image tools, for example:

```

magick creature_color.ppm creature_color.png

```

## References

- [1] J. Locke, *An Essay Concerning Human Understanding*. 1689.
- [2] F. Nietzsche, *The Gay Science*. 1882.
- [3] A. Camus, *The Myth of Sisyphus*. Gallimard, 1942.
- [4] S. Freud, “The Uncanny,” 1919.
- [5] D. Parfit, *Reasons and Persons*. Oxford University Press, 1984.
- [6] J. Kristeva, *Powers of Horror: An Essay on Abjection*. Columbia University Press, 1982.
- [7] E. M. Stein and R. Shakarchi, *Fourier Analysis: An Introduction*. Princeton University Press, 2003.

- [8] J. Bloomenthal, ed., *Introduction to Implicit Surfaces*. Morgan Kaufmann, 1997.
- [9] S. Osher and R. Fedkiw, *Level Set Methods and Dynamic Implicit Surfaces*. Springer, 2003.
- [10] R. J. Adler and J. E. Taylor, *Random Fields and Geometry*. Springer, 2007.
- [11] I. Daubechies, *Ten Lectures on Wavelets*. SIAM, 1992.
- [12] S. Mallat, *A Wavelet Tour of Signal Processing*. Academic Press, 3rd ed., 2009.
- [13] Y. Meyer, *Wavelets and Operators*. Cambridge University Press, 1992.
- [14] J.-L. Starck, F. Murtagh, and J. Fadili, *Sparse Image and Signal Processing: Wavelets and Related Geometric Multiscale Analysis*. Cambridge University Press, 2015.